Phase 6: Live Order Book (snapshot + diffs + sequence-gap resync)

What we built

- A **live order book** for the selected coin: red **asks** on top, green **bids** below, the **mid price and spread** in the middle
- **Depth bars** behind each row showing the cumulative size, so you can spot big "walls"
- It follows Binance's documented algorithm **"How to manage a local order book correctly"**: a REST snapshot, buffered WebSocket diffs, and **update-id sequence checks**
- **Gap detection → automatic resync**, with a **"Simulate gap"** button to watch it happen, plus a footer showing the book's update id and the resync count
- A hand-built **sorted map** (`BookSide`) using **binary search**, since JavaScript has no TreeMap

Files and what each one does

File	Role
`src/services/BookSide.ts`	One side of the book, always sorted best-first (Map + sorted array + binary search)
`src/services/LocalOrderBook.ts`	Bids + asks + `lastUpdateId` . `applyDiff()` returns `applied` / `stale` / `gap`
`src/hooks/useOrderBook.ts`	**The algorithm**: stream → buffer → snapshot → replay → live → resync on gap
`src/services/binanceRest.ts`	+ `fetchDepth()` (and a shared `getJson<T>()` helper with host fallback)
`src/services/binanceMessages.ts`	+ depth snapshot and `depthUpdate` parsers
`src/components/OrderBook.tsx`	The ladder UI: asks, spread row, bids, depth bars, footer

Concepts (theory only)

1. What an order book is

The list of all **resting limit orders** (Phase 5) for one market, grouped by price: - **Bids** = buy orders. The **best bid** is the **highest** price someone will pay. - **Asks** (offers) = sell orders. The **best ask** is the **lowest** price someone will sell at. - **Spread** = best ask – best bid. Small spread = a **liquid** market (BTC's is often just one tick: 0.01). - **Mid price** = (best bid + best ask) / 2. - **Depth** = how much quantity sits at each level. **Cumulative depth** answers "how much could I buy before the price moves past X?" -

A **market buy** eats the asks from the lowest price upwards. If it's bigger than the best level, it fills at worse prices: that's **slippage** (Phase 5's simplification, now visible).

2. Snapshot + diffs

Sending the whole book (thousands of levels) 10 times a second would be wasteful. So: - **Snapshot** (REST `GET /api/v3/depth`): the full book once, with a **version number** `lastUpdateId` - **Diffs** (WebSocket `btcusdt@depth@100ms`): every 100ms, only the **levels that changed**, as `[price, newQuantity]` - quantity **0** means the level is now **empty**, so delete it - otherwise it's the **new absolute quantity** (not "+5"), so you just set it - Same pattern as the chart in Phase 4 (history over REST + live over WS), but here the order and continuity of messages **matter**.

3. Sequence numbers & gap detection

Every diff carries `U` (first update id) and `u` (final update id). Our book is at version `lastUpdateId`:

Condition	Meaning	Action
`u ≤ lastUpdateId`	Already included	Ignore (**stale**)
`U > lastUpdateId + 1`	Versions in between are **missing**	**Gap**: the book is wrong, so resync
otherwise	It continues our version	Apply the levels, then `lastUpdateId = u`

Why a gap is fatal: diffs are *absolute* values for *some* levels. If you miss one, a level that should have been deleted stays forever (a "ghost" level), or a size is wrong, and nothing later will fix it. You can't patch it. You must **start over** from a new snapshot. Gaps happen on **reconnects** (messages sent while you were offline are gone), dropped messages, or a slow client. Our reconnect logic from Phase 2 needs no special code: the first diff after reconnecting shows a gap → automatic resync.

4. Binance's algorithm, step by step (`useOrderBook.ts`)

1. Open the diff stream and **buffer** the diffs.
2. When the first one arrives, **fetch the snapshot**.
3. If `snapshot.lastUpdateId < firstBufferedDiff.U`, the snapshot is **older** than our buffer (a hole between them) → fetch again.
4. Load the snapshot. Replay the buffer (stale diffs are skipped). The first applied diff must "straddle" the snapshot: `U ≤ lastUpdateId + 1 ≤ u`.
5. Apply new diffs live.
6. On a **gap** → go back to step 2.

Why buffer first? The REST call takes time. Diffs that happen *while* it's in flight must not be lost. Buffering them, then discarding the ones the snapshot already includes, gives a seamless join.

5. Why sorted maps (and binary search)

The UI keeps asking for the **best N levels** on each side. Options: | Approach | Update | "Top N" | |–|–|–| | Unsorted Map + sort on every render | O(1) | O(n log n) every time | | **Sorted structure** | O(log n) to find the spot | O(N): just take the first N |

Java has `TreeMap` (a red-black tree, with `firstKey()`, `headMap()`…). JavaScript has no built-in sorted map, so `BookSide` uses: - a `Map<price, qty>` for O(1) "does this level exist?" - an **array of prices kept sorted** (bids descending, asks ascending) - **binary search** to find where a price belongs: ~11 comparisons for 2000 levels, instead of 2000. (`splice` still shifts the array elements after it, which is fine for a few thousand items. A tree or skip list would make that O(log n) too.)

6. Performance: keep the big data outside React

The full book (often 1000+ levels per side) lives in a **plain object**. React only gets the **top 12 levels**, at most **once per animation frame** (Phase 3's `FrameBatcher` again). 10 diffs/sec × 2 sides × many levels, and React does at most one small render per frame.

7. Smaller things in this phase

- **Local state, not the global store:** only the order book panel uses this data, so it lives in a hook's `useState`.
- `key={symbol}` remounts the book when the coin changes, so it never briefly shows the old coin's prices. (Third use of this trick: price flash, order form, order book.)
- **Inline styles for data-driven values:** the depth bar width is `style={{ width: '37%' }}`. Tailwind classes are generated at build time, so they can't hold arbitrary runtime numbers.
- **Generics:** `getJson<T>(path)` lets each caller say what JSON shape it expects (like Java generics).
- **Known limitation:** the snapshot has 1000 levels per side. Levels deeper than that and never changed won't appear. That's irrelevant for the top 12 shown, but a full-depth display would use `limit=5000`.

Try it yourself

1. Watch the book update ~10 times a second. Levels appear, disappear and change size. The spread on BTC is usually 0.01.
2. Compare the mid price with the watchlist price: they stay very close.
3. Click **Simulate gap**: the next diff is "lost", the footer's **resync count** goes up, you see "Syncing…" for a moment, then the book returns. That's gap detection working.
4. Click **Simulate drop** in the header: the main socket reconnects (Phase 2). Each panel's socket is separate, so the order book's own socket isn't affected by that button.
5. DevTools → Network → WS → the `depth@100ms` connection: look at `U`, `u`, `b`, `a`. Check that each message's `U` = previous `u` + 1.
6. Switch coins: "Syncing…", then the new book (snapshot + buffer replay).

Resume mapping

> “**incremental order book** with **sorted maps** and **sequence-gap resync**” → `LocalOrderBook.ts`, `BookSide.ts`, `useOrderBook.ts`

Interview Qs

1. **How do you keep a local order book in sync?** Buffer WebSocket diffs, fetch a REST snapshot, drop diffs older than the snapshot, check the first one straddles it, apply the rest in order, and resync on any sequence gap.
2. **Why not just use the diffs?** They only contain *changed* levels. Without a snapshot you’d never learn about levels that didn’t change.
3. **How do you detect a missed message?** Update ids. If the next diff’s first id is greater than my last id + 1, something is missing.
4. **What do you do on a gap, and why not just continue?** Discard and rebuild from a new snapshot. Diffs are absolute per level, so a missed deletion leaves a ghost level that no later message will fix.
5. **Why buffer before the snapshot?** Updates that happen during the REST round-trip would otherwise be lost. The buffer plus the update ids join the snapshot and the stream with no hole.
6. **Data structure for a book side?** A sorted map keyed by price (a TreeMap in Java). In JS: a Map plus a sorted price array with binary-search insert/delete. Best price = first element, and top N = slice.
7. **Bid vs ask, spread, mid?** Bid = highest buy price, ask = lowest sell price, spread = ask – bid, mid = their average.
8. **How do you stop the book from killing performance?** The full book lives outside React, and only the top N levels are published, at most once per animation frame.
9. **What happens on a reconnect?** The first diff after reconnecting shows a gap, which triggers a resync automatically. No special-case code needed.